



in collaboration with



Assignment Title:

Lab 4:

Investigating Process Lifecycles and OS Interaction.

Course name:

***BSc. (Hons) Computer Science with AI, Software College of IT and E-Commerce,
Coventry University***

CourseWork Type:

Individual

Module Name:

ST5003CMD Operating Systems, Security, and Networks

Intake: September

Submitted By:

CU ID: 16536799

College ID: 250073

Name: Ashish Shrestha(A)

Submitted To:

Rupak Rajbanshi

TABLE OF CONTENTS:

I. INTRODUCTION:	1
II. PRIMARY DATA TYPES AND OS ALLOCATION:	2
A. Data Type Sizes Across Architectures:	2
B. Memory Allocation and Data Types:	3
III. Process Lifecycle:	4
A. Process Creation:	4
B. Process Execution:	4
C. Process Termination:	5
IV. OS Interaction:	5
A. System Calls:	5
B. Process Scheduling:	6
C. Inter-Process Communication:	6
V. Laboratory Tasks and Observations:	6
VI. Knowledge Check Questions:	7
A. C Source Code (Data Type Size):	7
III. OS MEMORY SEGMENTS:	8
IV. PRACTICAL CODE EXAMPLES:	9
A. Lab 1: Data Type Size Exploration:	10
B. Lab 2: Memory Segment Address Mapping:	12
VI. SUMMARY AND KEY TAKEAWAYS:	14
VII. CONCLUSION:	15
Knowledge Check Questions:	16

LIST OF FIGURES:

Fig1: Memory management in C program:	1
Fig2: Memory management in C program:	2
Fig3: Table that shows the size of all datatypes in C program:	3
Fig4: Process life cycle diagram:	4
Fig5: C source code which shows the sizes of all data types:	7
Fig6: Command that used to check the output of C program which checks the size of Data type in C:	7
Fig7: Command that used to create an object in C program file data_type_size.c file:	8
Fig8: Command that used to check output in C program file data_type_size.c using object ./data_type_size:	8
Fig9: Table for Memory Segment Overview:	8
Fig10: Memory Layout Visualization:	9
Fig11: C source code to Observe Variable Addresses:	9
Fig12: Command that used to observing_virtual_addresses:	10
Fig13: Command that is used to create an object in C program observing_virtual_addresses.c file:	10
Fig14: Command that is used to observe virtual addresses:	10
Fig15: C source code for pointer and memory allocation:	11
Fig16: Command that is used to show memory address in C program:	11
Fig17: Command that is used to create an object in C program pointer_and_memory_allocation.c file:	11
Fig18: Command that is used to check the virtual_addresses and memory allocation in C program:	11
Fig19: C source code which allocates sizes of different datatypes:	12
Fig20: Command that is used to allocate the size of different datatypes:	12
Fig21: Command that is used to create objects in C program using gcc datatype_size.c file:	12
Fig22: Command that is used to allocate the size of different datatypes:	13
Fig23: Table which shows different sizes of datatypes:	13
Fig24: C source code which uses Memory Segment Address Mapping:	13
Fig25: Command that is used to show output of memory_segment of different data types in C:	14
Fig26: gcc Command that is used to create objects on a C program using gcc memory_segments.c -o memory_segments:	14
Fig27: Command that is used to show memory address of different datatypes in C program:	14

I. INTRODUCTION:

Data types form the foundation of programming and operating system interaction. A data type is a contract between programmer, compiler, and operating system that specifies:

- 1. The number of bytes to allocate in memory.
- 2. How to interpret those bytes (integer, character, floating-point).
- 3. What operations are valid on that data.

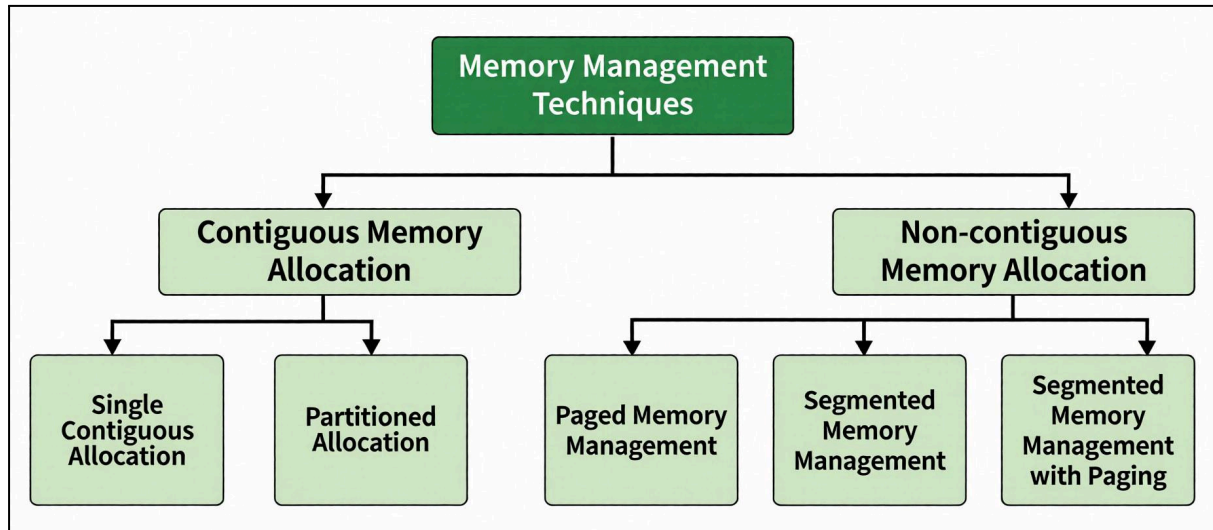


Fig1: Memory management in C program.

When a programmer declares a variable, the OS must allocate physical or virtual memory to store that data. Understanding this relationship is crucial for writing efficient, secure, and portable code.

Summary:

Data types establish an agreement among programmers, compilers, and the operating system on the amount of memory, interpretation of bytes, and operations possible.

Given that declaration of a variable allocates memory, it becomes imperative to understand data types for writing effective C code.

II. PRIMARY DATA TYPES AND OS ALLOCATION:

A. Data Type Sizes Across Architectures

Table I shows how data type sizes vary across different system architectures, directly impacting OS memory allocation.

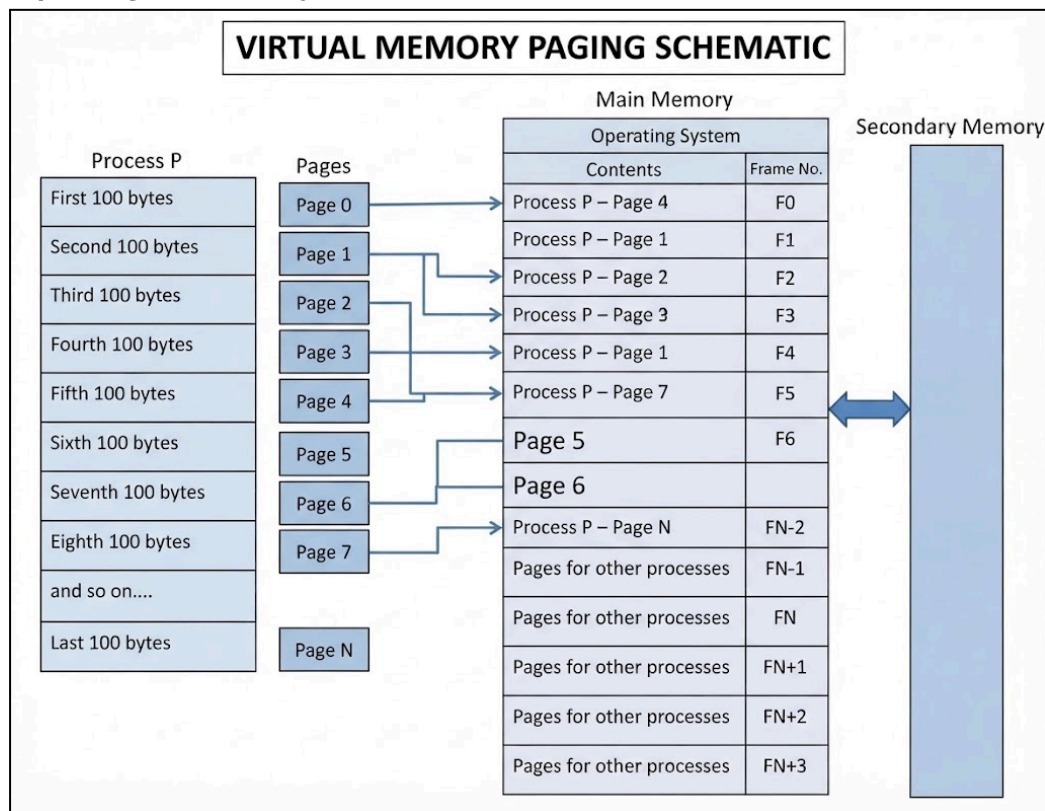


Fig2: Memory management in C program.

II. Primary Data Types and OS Allocation:

The OS allocates memory based on the declared data type.

This allocation affects efficiency, security, and portability.

A. Data Type Sizes Across Architectures:

Data type sizes vary across 32-bit and 64-bit architectures.

This variation directly impacts OS memory allocation and portability.

B. Memory Allocation and Data Types:

Static variables are allocated at compile time; dynamic variables at runtime.

The OS manages virtual memory and maps it to physical memory.

Summary:

Section II describes how C data types dictate how memory allocation is done by the OS in terms of size differences between architectures, particularly in cases of int, long, and pointer types.

Static variables are assigned at compile time while dynamic variables are assigned at runtime. The OS performs the virtual to physical memory mapping operation.

TABLE 1:

DATA TYPE SIZES ACROSS SYSTEM ARCHITECTURES				
Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

Fig3: Table that shows the size of all datatypes in C program.

Summary:

The sizes of various data types in the C programming language on 16-bit, 32-bit, and 64-bit machine systems is provided in Table 1. Some of the data types have varying sizes while others have constant sizes.

The above mentioned differences affect various areas such as memory allocation by operating system, pointer arithmetic, 64-bit addressing, file offsets, and accuracy.

III. Process Lifecycle:

Processes can exist in various stages: new, ready, running, waiting, and terminated. These stages are managed by the process scheduler.

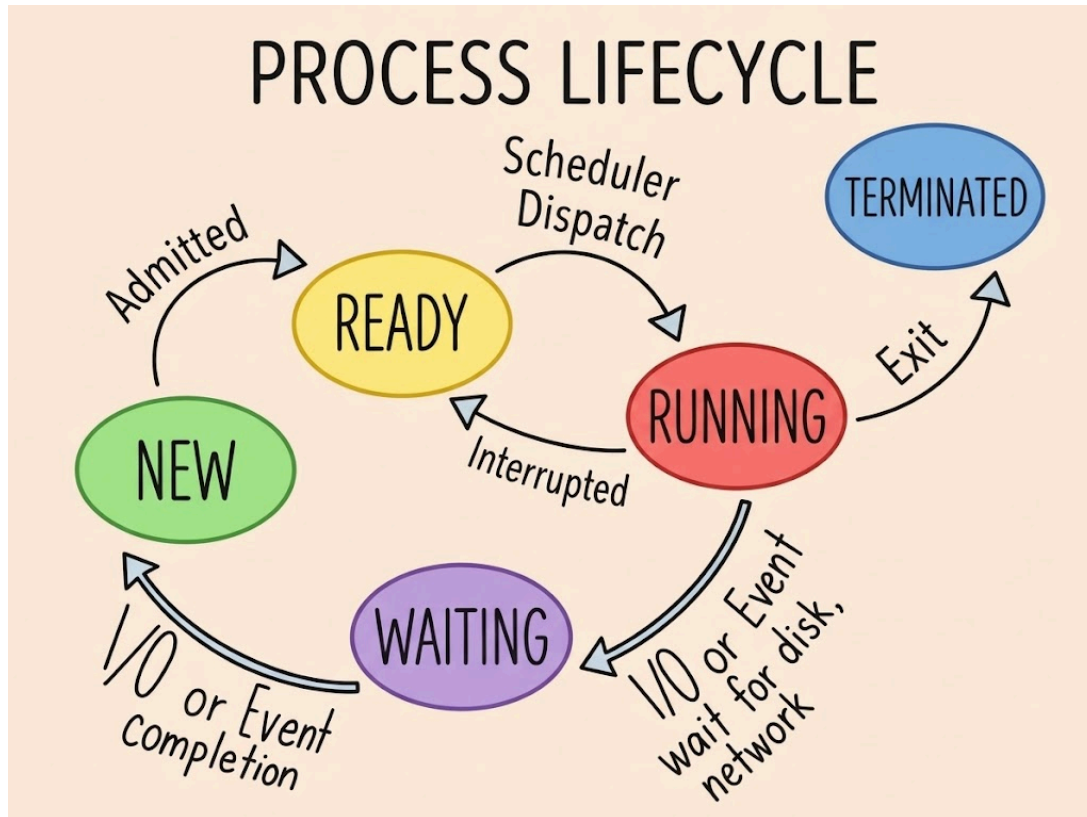


Fig4: Process life cycle diagram.

A. Process Creation

A parent process creates a child process by using `fork()`.

The child process gets an exact copy of the parent process address space.

B. Process Execution

Using `exec()`, a new program is executed replacing the process image.

This enables processes to execute other programs.

C. Process Termination

Using `exit()`, a process terminates and gives a status back to the OS.

Using `wait()`, a parent can wait for its child to terminate.

Summary:

The processes go through various stages like new, ready, running, waiting, and terminated states that are controlled by the process scheduler.

The creation is done using `fork()`, execution using `exec()`, termination using `exit()`, and synchronization using `wait()`.

IV. OS Interaction:

- *Interaction with the OS occurs using system calls.*
- **Examples include:**
fork, exec, wait, exit, open, read, and write.

A. System Calls

- *System calls transition from user space to kernel space.*
- *They offer a way to access the hardware and OS functionality.*

B. Process Scheduling

- *The scheduler determines which process gets scheduled next.*
- *Its goal is to ensure maximum CPU usage and fairness.*

C. Inter-Process Communication

- *Pipes, shared memory, and message queues enable inter-process communication.*
- *OS ensures proper synchronization to prevent any race conditions.*

V. Laboratory Tasks and Observations:

Task 1: Check the running processes with ps or top:

- *This reveals the process ID, state, and resources consumed by each process.*

Task 2: Make a child process with fork().

- *Both processes work simultaneously and output separate PIDs.*

Task 3: Replaces the process image with exec().

- *The original program gets substituted with a new program.*

Task 4: Synchronization of parent and child with wait().

- *The parent waits for the child to terminate execution.*

Task 5: Use exit status with echo \$?

- *A value of zero indicates success, non-zero denotes failure.*

VI. Knowledge Check Questions

Q1: What is a process?

- *A process is a running program that has its own memory and state.*

Q2: What is the distinction between fork() and exec()?

- *fork() creates a child process while exec() replaces the current process.*

Q3: Why does the OS require system calls?

- *They allow for safe interaction with the kernel from user space.*

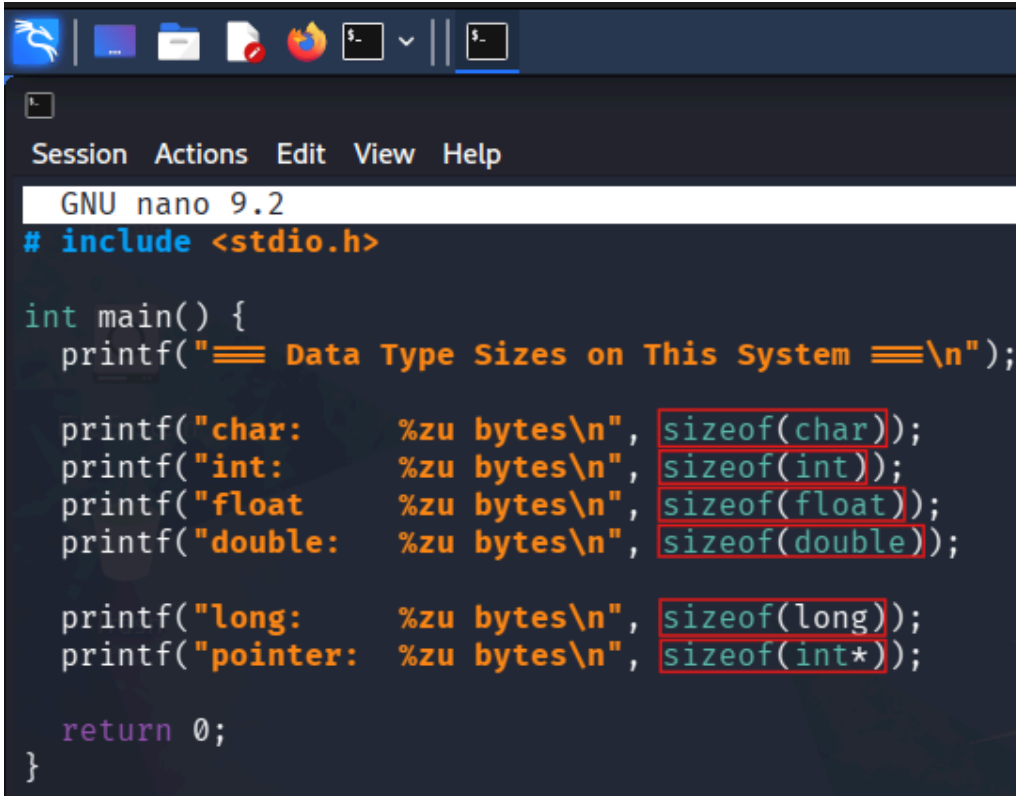
Summary:

Sections IV-VI discuss OS interactions through system calls (fork, exec, wait, exit, open, read, and write), process scheduling, and IPC through pipes, shared memory, and message queues.

The Lab exercises include ps/top, fork(), exec(), wait() operations, and status of exit, whereas the knowledge check covers topics like processes, fork versus exec, and system calls requirement.

A. C Source Code:

We create a program that shows the size of all datatypes.

A screenshot of a terminal window with a dark blue background. At the top, there is a taskbar with icons for a terminal, a file manager, a web browser, and a terminal window. Below the taskbar is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The main area shows the GNU nano 9.2 editor. The code is written in a monospaced font with syntax highlighting: preprocessor directives and keywords are in blue, strings are in orange, and function names are in green. The code defines a main function that prints the sizes of various data types using printf and sizeof. The sizeof expressions are highlighted with red boxes. The code is as follows:

```
GNU nano 9.2
# include <stdio.h>

int main() {
    printf("≡ Data Type Sizes on This System ≡\n");

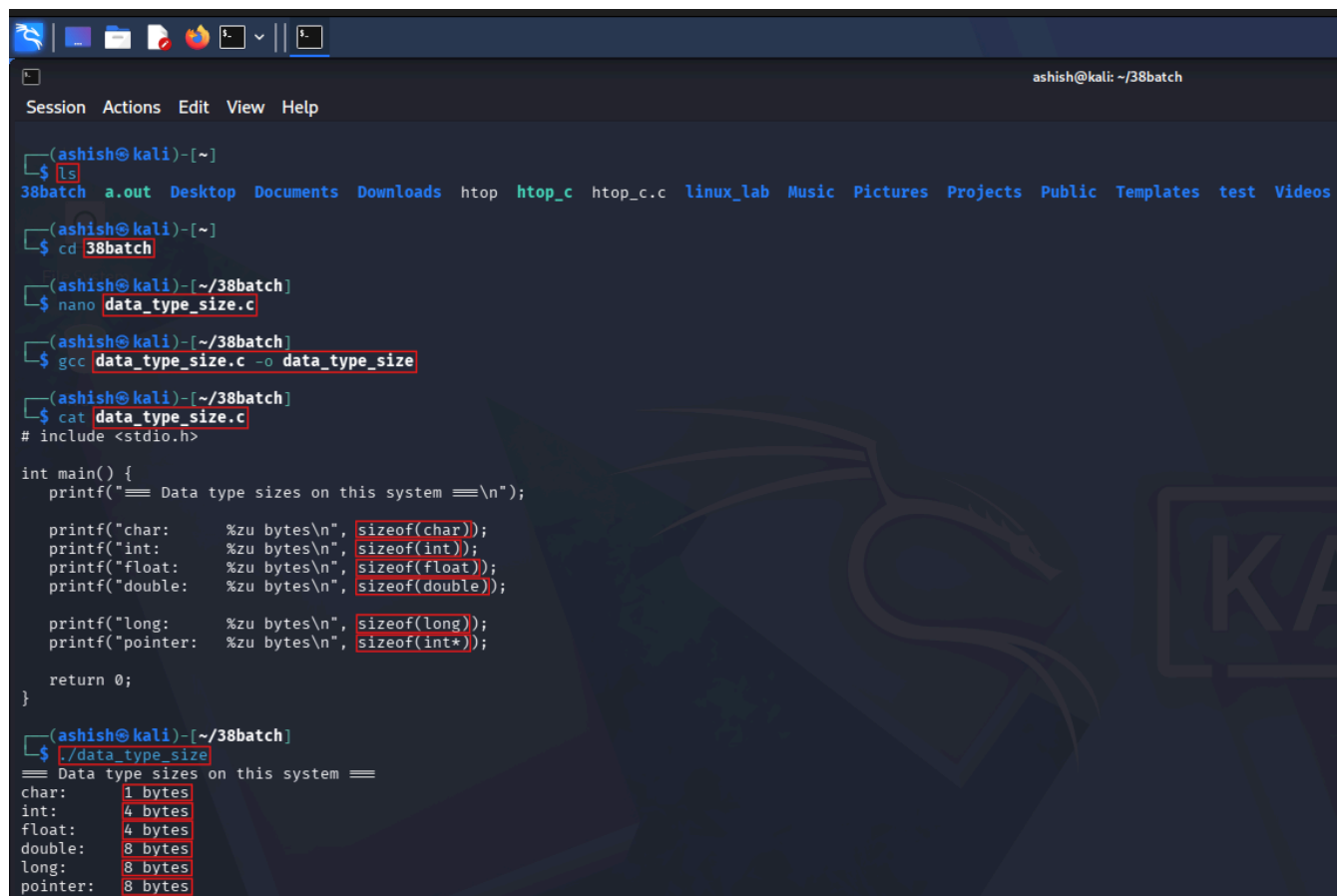
    printf("char:      %zu bytes\n", sizeof(char));
    printf("int:       %zu bytes\n", sizeof(int));
    printf("float    %zu bytes\n", sizeof(float));
    printf("double:   %zu bytes\n", sizeof(double));

    printf("long:     %zu bytes\n", sizeof(long));
    printf("pointer:  %zu bytes\n", sizeof(int*));

    return 0;
}
```

Fig5: C source code which shows the sizes of all data types.

1. File Operation: This command returns Success for positive input and fail for negative input values.



```
(ashish@kali)-[~]
$ ls
38batch  a.out  Desktop  Documents  Downloads  htop  http_c  http_c.c  linux_lab  Music  Pictures  Projects  Public  Templates  test  Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano data_type_size.c

(ashish@kali)-[~/38batch]
$ gcc data_type_size.c -o data_type_size

(ashish@kali)-[~/38batch]
$ cat data_type_size.c
# include <stdio.h>

int main() {
    printf("== Data type sizes on this system ==\n");

    printf("char:      %zu bytes\n", sizeof(char));
    printf("int:        %zu bytes\n", sizeof(int));
    printf("float:     %zu bytes\n", sizeof(float));
    printf("double:    %zu bytes\n", sizeof(double));

    printf("long:      %zu bytes\n", sizeof(long));
    printf("pointer:   %zu bytes\n", sizeof(int*));

    return 0;
}

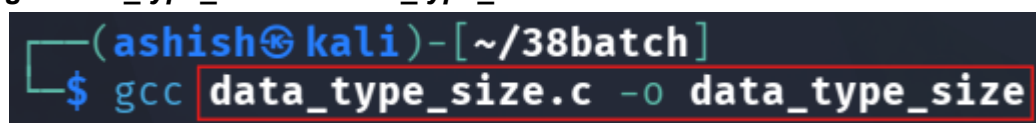
(ashish@kali)-[~/38batch]
$ ./data_type_size
== Data type sizes on this system ==
char:      1 bytes
int:       4 bytes
float:     4 bytes
double:    8 bytes
long:      8 bytes
pointer:   8 bytes
```

Fig6: Command that used to check the output of C program which checks the size of Data type in C.

B. Terminal Execution & Verification:

➤ GCC command to create object:

gcc data_type_size.c -o data_type_size



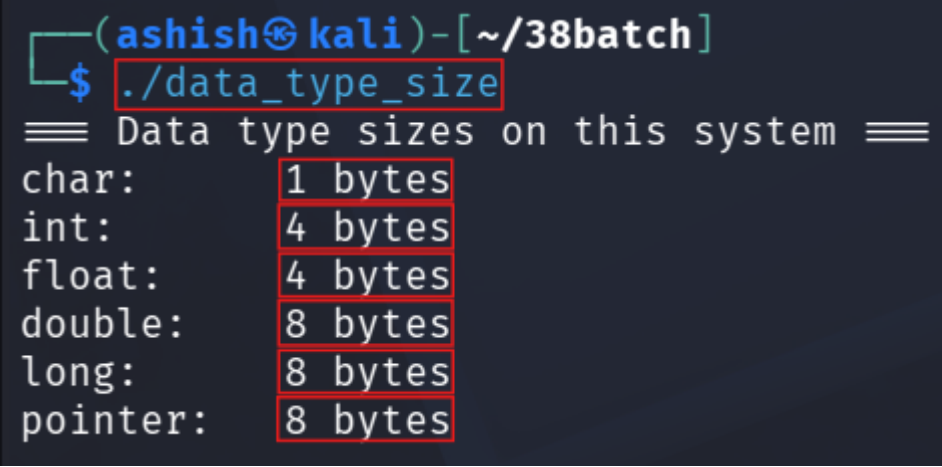
```
(ashish@kali)-[~/38batch]
$ gcc data_type_size.c -o data_type_size
```

Fig7: Command that used to create an object in C program file **data_type_size.c** file.

C. Check output for size of datatypes in C:

➤ Command to check output of C program datatypes:

`./data_type_size`



```
(ashish@kali)-[~/38batch]
$ ./data_type_size
≡ Data type sizes on this system ≡
char:      1 bytes
int:       4 bytes
float:     4 bytes
double:    8 bytes
long:      8 bytes
pointer:   8 bytes
```

Fig8: Command that used to check output in C program file `data_type_size.c` using object `./data_type_size`.

Summary:

In this part, we have an example of C code which returns the sizes of various data types where functions related to file operations return whether it was successful or not depending on the input parameters.

This code is compiled using the command `gcc data_type_size.c -o data_type_size` and executed with the command `./data_type_size`.

III. OS MEMORY SEGMENTS:

The operating system creates a process address space, a virtual view of memory divided into segments. Each segment serves a specific purpose and has different characteristics.

A. Memory Segment Overview.

PROCESS MEMORY SEGMENTS AND THEIR PURPOSES			
Segment	Purpose	Grows	Managed By
Text	Machine code (instructions)	Fixed	Compiler/Loader
Data	Initialized global/static variables	Fixed	Compiler/OS
BSS	Uninitialized global/static variables	Fixed	OS
Heap	Dynamically allocated memory	Upward	Programmer + OS
Stack	Local variables, function calls	Downward	Compiler + OS

Fig9: Table for Memory Segment Overview.

B. Memory Layout Visualization.

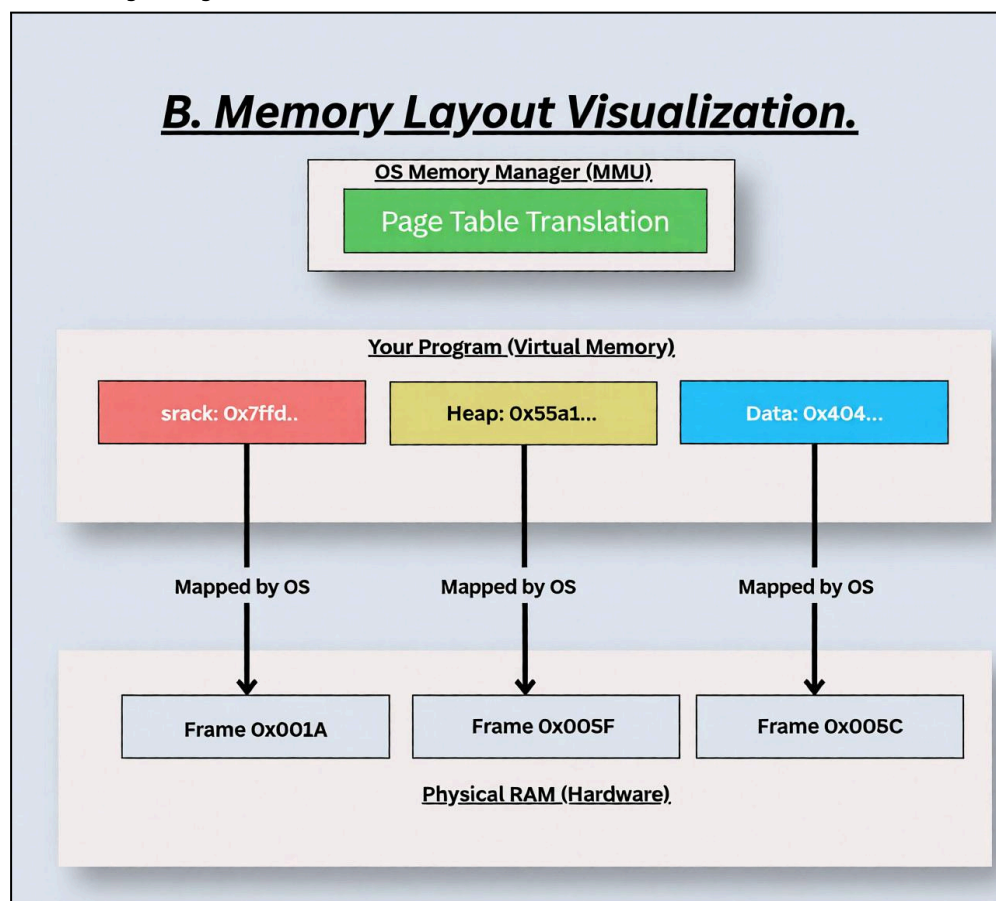


Fig10: Memory Layout Visualization.

Summary:

The operating system creates a separate virtual address space for each process by allocating segments (text, data, BSS, heap, and stack), each with its own purpose and addressing scheme stack highest, heap middle range, and data/BSS lowest.

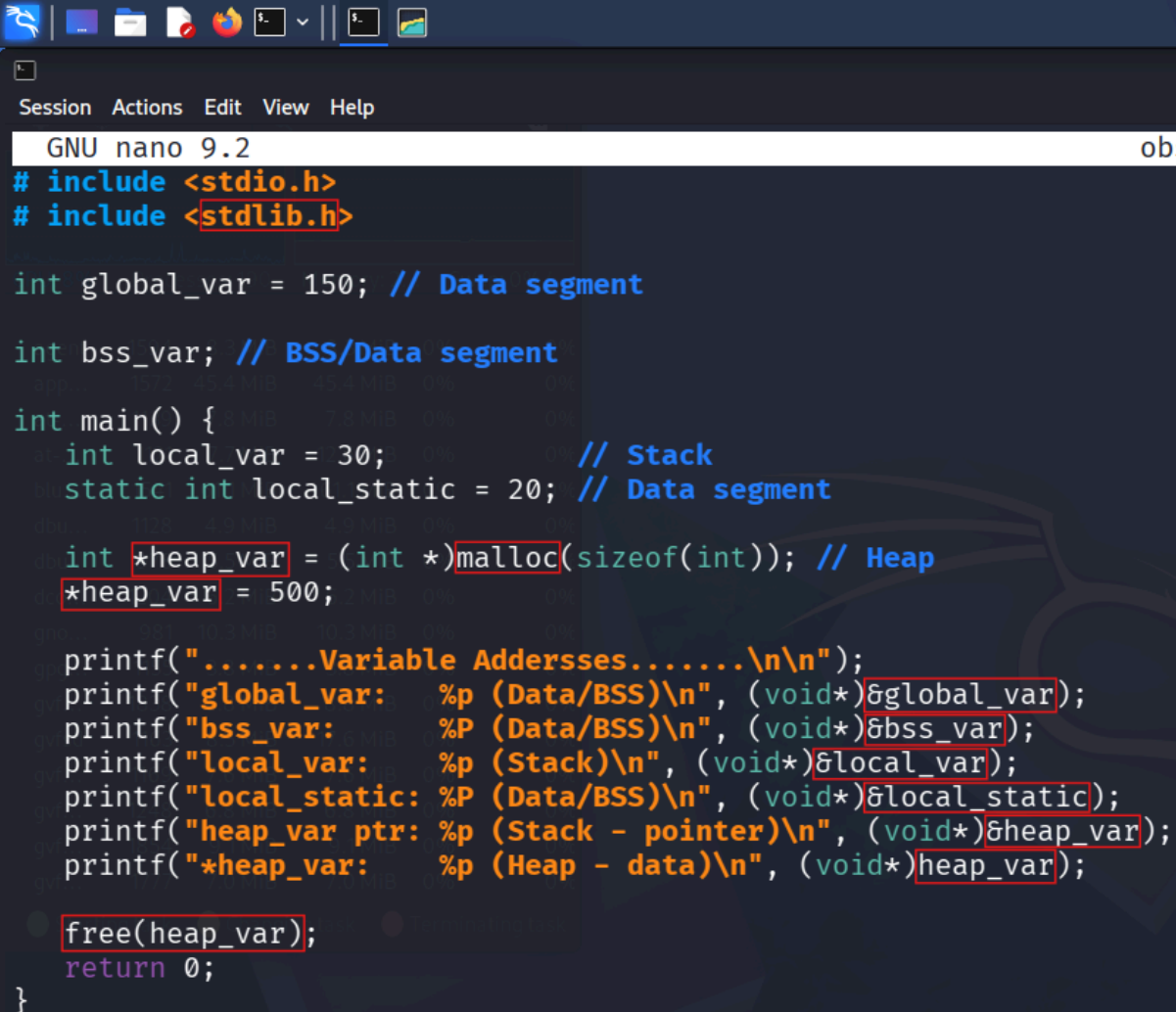
Examples of printing addresses of variables in practical C code illustrate this memory organization.

Key Observation:

Stack addresses are high (0x7ffd...), Heap addresses are mid-range (0x55a1...), and Data/BSS addresses are low (0x404...).

IV. PRACTICAL CODE EXAMPLES:

A. Observing Variable Addresses:



```
GNU nano 9.2 ob:
# include <stdio.h>
# include <stdlib.h>

int global_var = 150; // Data segment

int bss_var; // BSS/Data segment

int main() {
    int local_var = 30; // Stack
    static int local_static = 20; // Data segment

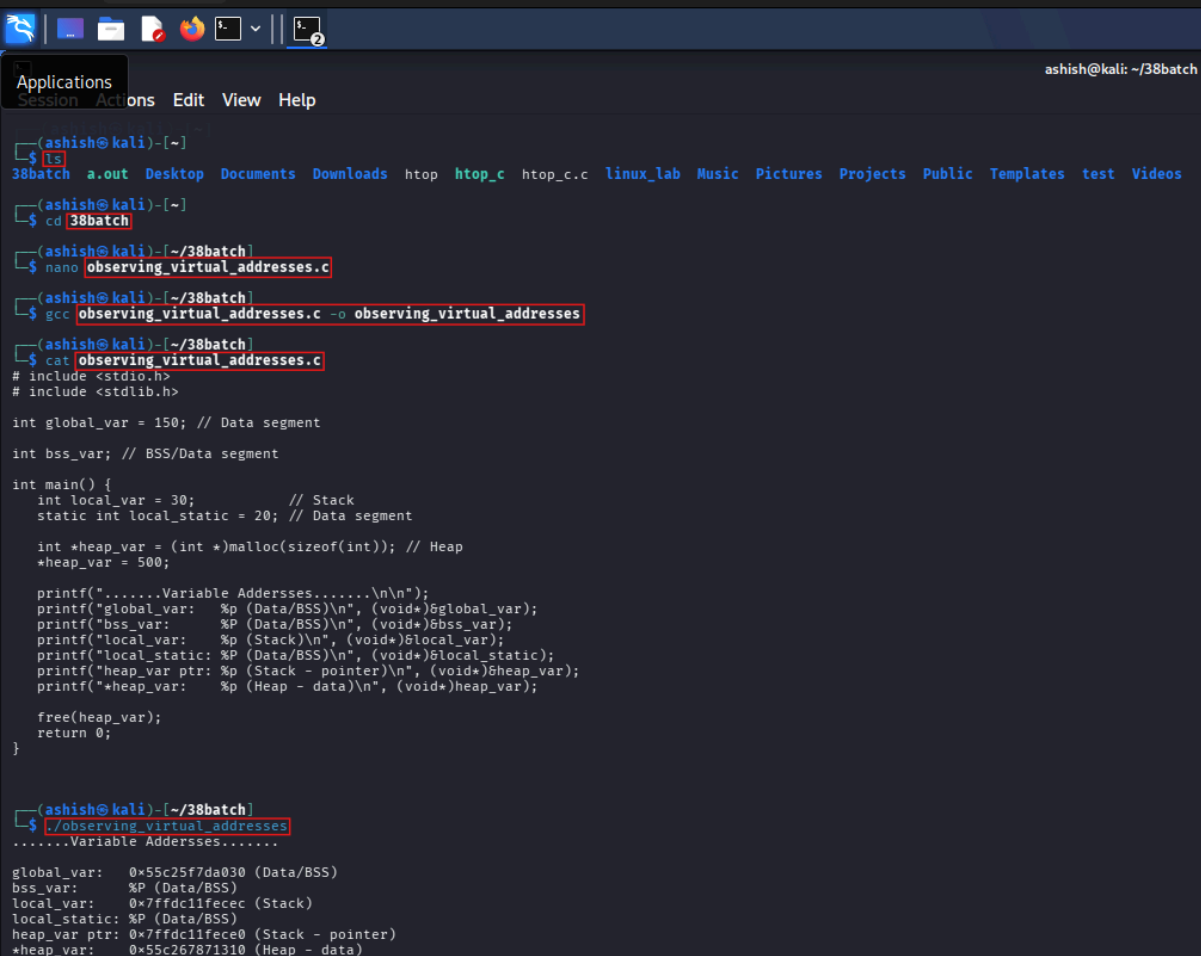
    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf(".....Variable Addersses.....\n\n");
    printf("global_var:    %p (Data/BSS)\n", (void*)&global_var);
    printf("bss_var:         %P (Data/BSS)\n", (void*)&bss_var);
    printf("local_var:       %p (Stack)\n", (void*)&local_var);
    printf("local_static:   %P (Data/BSS)\n", (void*)&local_static);
    printf("heap_var ptr:   %p (Stack - pointer)\n", (void*)&heap_var);
    printf("*heap_var:     %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}
```

Fig11: C source code to Observe Variable Addresses.

2. File Operation: This command is used to observe variable addresses.



```
(ashish@kali)-[~]
$ ls
38batch  a.out  Desktop  Documents  Downloads  htop  htop_c  htop_c.c  linux_lab  Music  Pictures  Projects  Public  Templates  test  Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano observing_virtual_addresses.c

(ashish@kali)-[~/38batch]
$ gcc observing_virtual_addresses.c -o observing_virtual_addresses

(ashish@kali)-[~/38batch]
$ cat observing_virtual_addresses.c
# include <stdio.h>
# include <stdlib.h>

int global_var = 150; // Data segment

int bss_var; // BSS/Data segment

int main() {
    int local_var = 30; // Stack
    static int local_static = 20; // Data segment

    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var: %p (Data/BSS)\n", (void*)&global_var);
    printf("bss_var: %p (Data/BSS)\n", (void*)&bss_var);
    printf("local_var: %p (Stack)\n", (void*)&local_var);
    printf("local_static: %p (Data/BSS)\n", (void*)&local_static);
    printf("heap_var ptr: %p (Stack - pointer)\n", (void*)&heap_var);
    printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}

(ashish@kali)-[~/38batch]
$ ./observing_virtual_addresses
.....Variable Addresses.....

global_var: 0x55c25f7da030 (Data/BSS)
bss_var: %P (Data/BSS)
local_var: 0x7ffdc11fec0c (Stack)
local_static: %P (Data/BSS)
heap_var ptr: 0x7ffdc11fec00 (Stack - pointer)
*heap_var: 0x55c267871310 (Heap - data)
```

Fig12: Command that used to observing_virtual_addresses.

B. Terminal Execution & Verification:

➤ GCC command to create object:

`gcc observing_virtual_addresses.c -o observing_virtual_addresses`



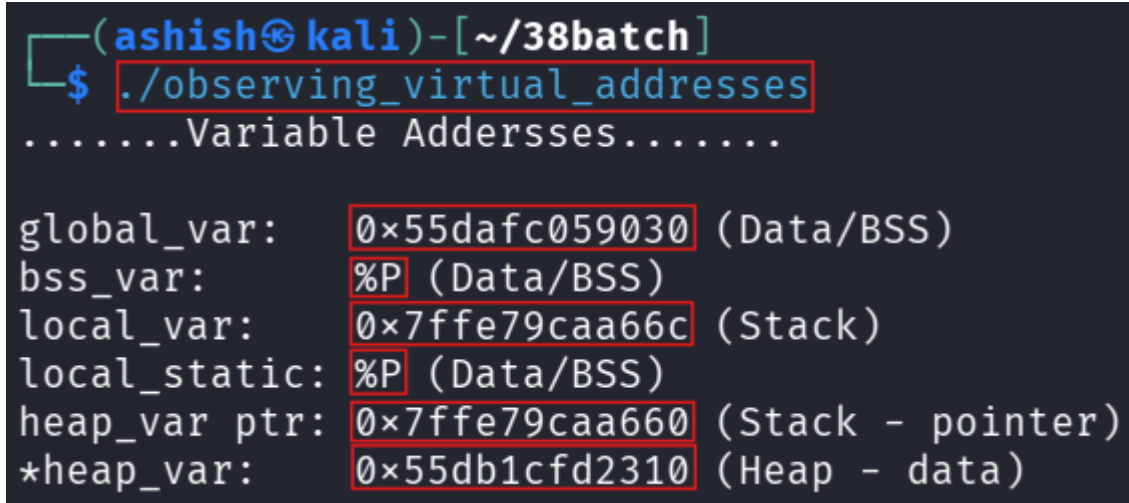
```
(ashish@kali)-[~/38batch]
$ gcc observing_virtual_addresses.c -o observing_virtual_addresses
```

Fig13: Command that is used to create an object in C program `observing_virtual_addresses.c` file.

C. Check the variable address in C:

➤ Gcc command to check virtual_addresses :

`./gcc observing_virtual_addresses.c -o observing_virtual_addresses`



```
(ashish@kali)-[~/38batch]
$ ./observing_virtual_addresses
.....Variable Addersses.....

global_var: 0x55dafc059030 (Data/BSS)
bss_var: %P (Data/BSS)
local_var: 0x7ffe79caa66c (Stack)
local_static: %P (Data/BSS)
heap_var ptr: 0x7ffe79caa660 (Stack - pointer)
*heap_var: 0x55db1cfd2310 (Heap - data)
```

Fig14: Command that is used to observe virtual addresses.

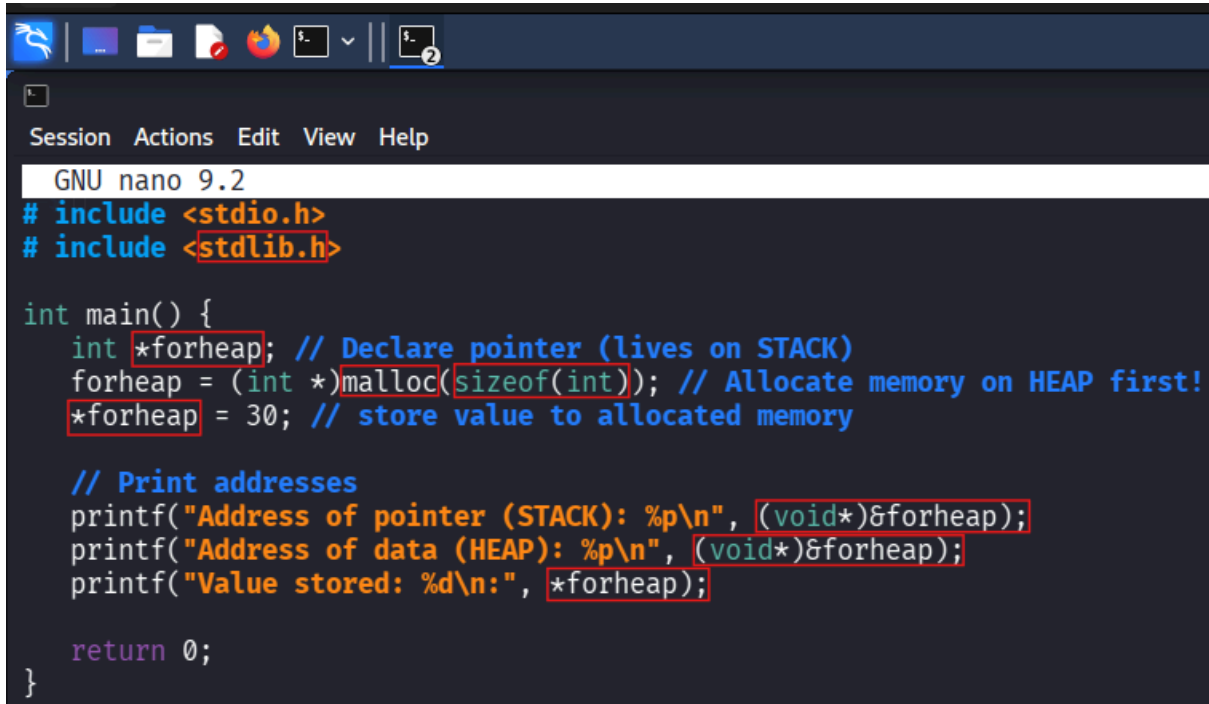
Summary:

The following section illustrates how a C program (`observing_virtual_addresses.c`) is compiled and run to print out the addresses of variables and examine the memory segments on the operating system.

To compile the program, type **`gcc observing_virtual_addresses.c -o observing_virtual_addresses`**. To execute the program, type `./observing_virtual_addresses` (and not `./gcc...`).

B. Pointer and memory allocation:

➤ C program source code which allocates memory address and using pointer.

A screenshot of a terminal window with a dark background. The window title bar shows standard Linux icons and a terminal icon. The menu bar includes 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal text shows the GNU nano 9.2 editor with C code. The code includes <stdio.h> and <stdlib.h>. The main function declares a pointer 'forheap' on the stack, allocates memory on the heap using 'malloc(sizeof(int))', stores the value 30, and prints the addresses of the pointer and data, along with the stored value. Red boxes highlight the pointer variable 'forheap', the 'malloc' function, 'sizeof(int)', the address cast '(void*)&forheap', and the dereferenced pointer '*forheap'.

```
GNU nano 9.2
# include <stdio.h>
# include <stdlib.h>

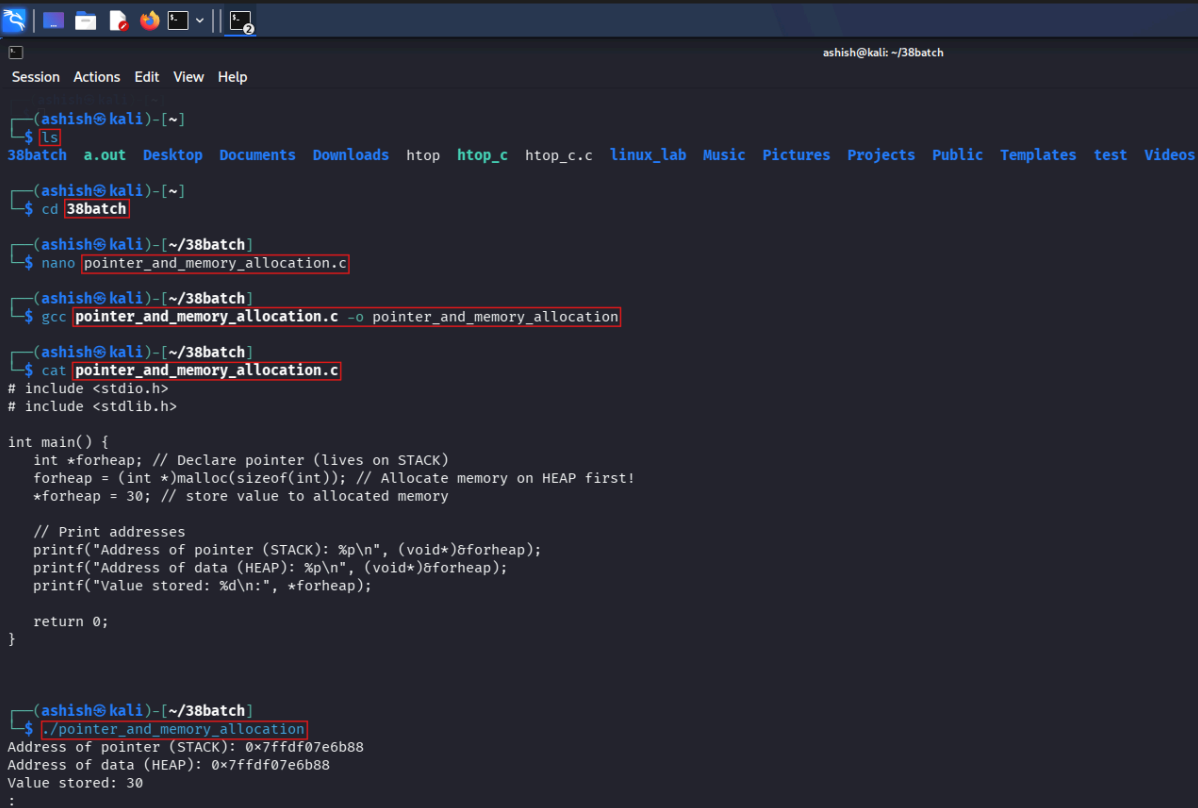
int main() {
    int *forheap; // Declare pointer (lives on STACK)
    forheap = (int *)malloc(sizeof(int)); // Allocate memory on HEAP first!
    *forheap = 30; // store value to allocated memory

    // Print addresses
    printf("Address of pointer (STACK): %p\n", (void*)&forheap);
    printf("Address of data (HEAP): %p\n", (void*)&forheap);
    printf("Value stored: %d\n:", *forheap);

    return 0;
}
```

Fig15: C source code for pointer and memory allocation.

3. File Operation: This command is used to observe variable addresses.



```
(ashish@kali)-[~]
$ ls
38batch a.out Desktop Documents Downloads htop htop_c htop_c.c linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano pointer_and_memory_allocation.c

(ashish@kali)-[~/38batch]
$ gcc pointer_and_memory_allocation.c -o pointer_and_memory_allocation

(ashish@kali)-[~/38batch]
$ cat pointer_and_memory_allocation.c
# include <stdio.h>
# include <stdlib.h>

int main() {
    int *forheap; // Declare pointer (lives on STACK)
    forheap = (int *)malloc(sizeof(int)); // Allocate memory on HEAP first!
    *forheap = 30; // store value to allocated memory

    // Print addresses
    printf("Address of pointer (STACK): %p\n", (void*)&forheap);
    printf("Address of data (HEAP): %p\n", (void*)&forheap);
    printf("Value stored: %d\n:", *forheap);

    return 0;
}

(ashish@kali)-[~/38batch]
$ ./pointer_and_memory_allocation
Address of pointer (STACK): 0x7ffdf07e6b88
Address of data (HEAP): 0x7ffdf07e6b88
Value stored: 30
:
```

Fig16: Command that is used to show memory address in C program.

B. Terminal Execution & Verification:

➤ GCC command to create object:

gcc pointer_and_memory_allocation.c -o pointer_and_memory_allocation



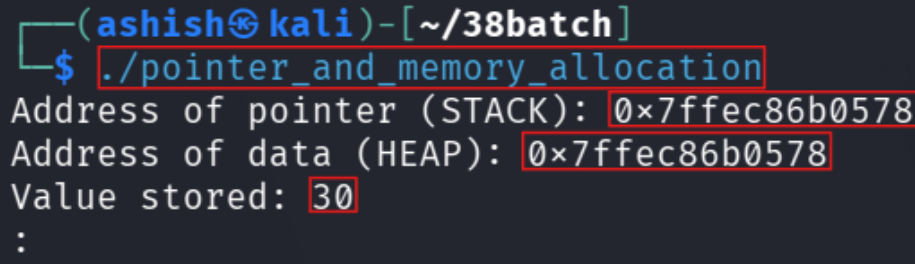
```
(ashish@kali)-[~/38batch]
$ gcc pointer_and_memory_allocation.c -o pointer_and_memory_allocation
```

Fig17: Command that is used to create an object in C program **pointer_and_memory_allocation.c** file.

C. Check the variable address in C:

➤ Gcc command to check virtual_addresses :

./gcc observing_virtual_addresses.c -o observing_virtual_addresses



```
(ashish@kali)-[~/38batch]
$ ./pointer_and_memory_allocation
Address of pointer (STACK): 0x7ffec86b0578
Address of data (HEAP): 0x7ffec86b0578
Value stored: 30
:
```

Fig18: Command that is used to check the virtual_addresses and memory allocation in C program.

Summary:

*The following is an example of a C program that will allocate memory and work with pointers to watch for addresses: (pointer_and_memory_allocation.c), compiled using **gcc***

pointer_and_memory_allocation.c -o pointer_and_memory_allocation.

Execute by: ./pointer_and_memory_allocation

A. Lab 1: Data Type Size Exploration:

Objective: Use `sizeof()` to observe how much memory

Each data type is used on your system.

Tasks:

1. Create a C program that prints the size of: `char`, `int`, `float`, `double`, `long`, `unsigned int`, `long`.
2. Compile and run on your system.
3. Compare output with Table I.
4. Answer: Why might `sizeof(long)` differ between 32-bit and 64-bit systems?

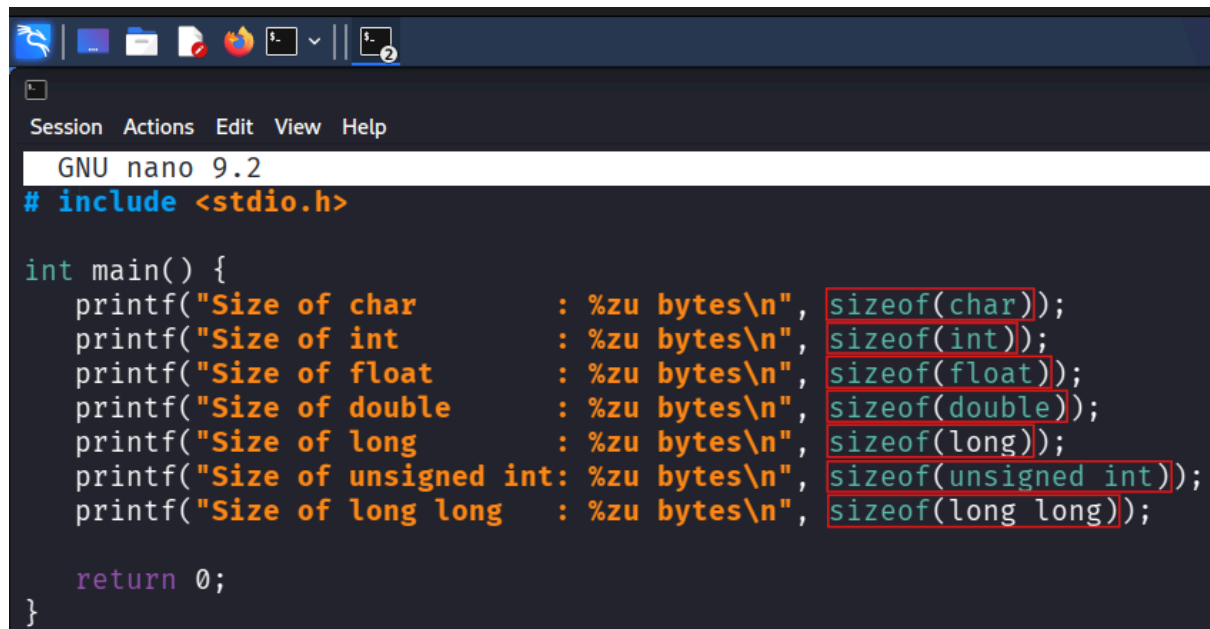
Deliverables:

- Source code file (.c)
- Screenshot of compilation (no errors)
- Screenshot of program output
- 1-2 sentences answering the question above

1. Create a C program that prints the size of: `char`, `int`, `float`, `double`, `long`, `unsigned int`, `long`.

A. C Source Code:

C program source code which allocates sizes of different datatypes.

A screenshot of a terminal window with a dark background. At the top, there's a taskbar with icons for a file manager, a web browser, and a terminal. The terminal window has a title bar with a close button and a tab labeled '2'. Below the title bar is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The main area of the terminal shows the GNU nano 9.2 editor. The code is written in a syntax-highlighted style: keywords like 'include', 'main', 'return' are in blue; strings are in orange; and the 'sizeof' function and its arguments are in green. The code prints the size of various data types in bytes. The 'sizeof' function calls are highlighted with red boxes.

```
GNU nano 9.2
# include <stdio.h>

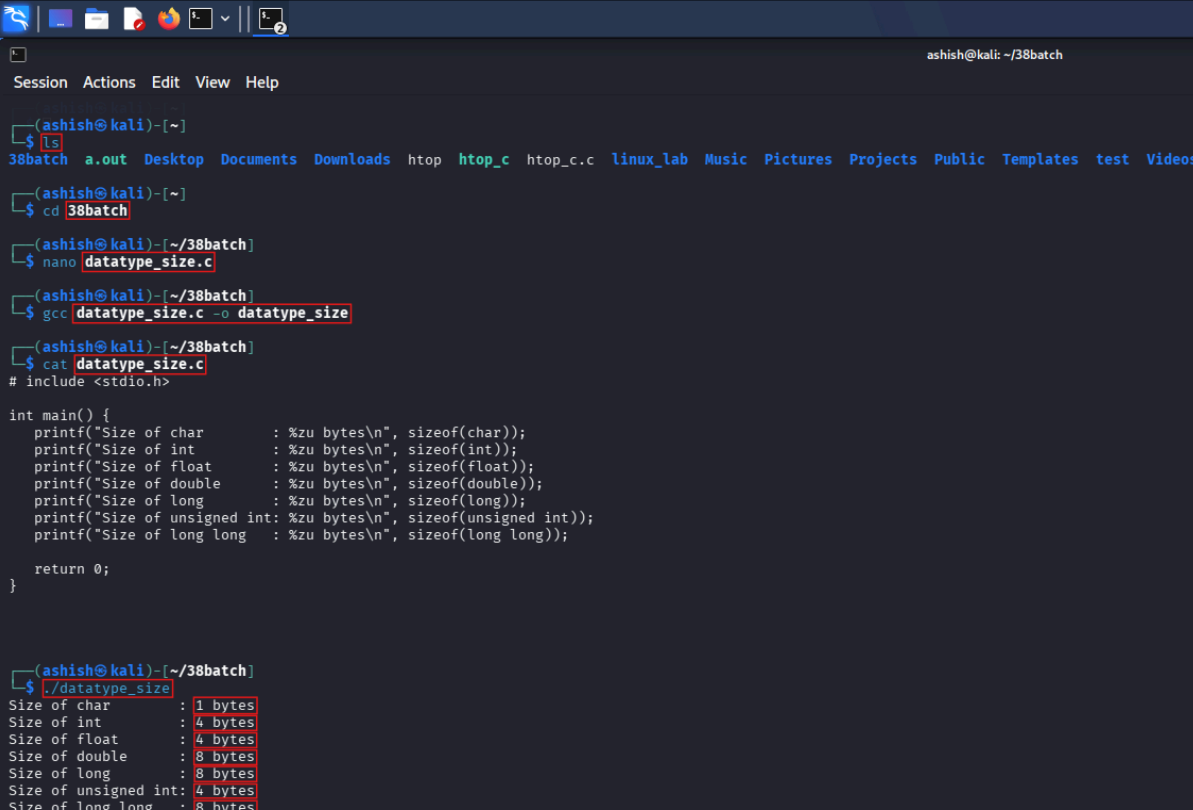
int main() {
    printf("Size of char      : %zu bytes\n", sizeof(char));
    printf("Size of int       : %zu bytes\n", sizeof(int));
    printf("Size of float      : %zu bytes\n", sizeof(float));
    printf("Size of double     : %zu bytes\n", sizeof(double));
    printf("Size of long        : %zu bytes\n", sizeof(long));
    printf("Size of unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("Size of long long   : %zu bytes\n", sizeof(long long));

    return 0;
}
```

Fig19: C source code which allocates sizes of different datatypes.

4. File Operation: This command is used to check output in C programs which allocate sizes of different datatypes.

2. Compile and run on your system.



```
(ashish@kali)-[~]
$ ls
38batch  a.out  Desktop  Documents  Downloads  http  http_c  http_c.c  linux_lab  Music  Pictures  Projects  Public  Templates  test  Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano datatype_size.c

(ashish@kali)-[~/38batch]
$ gcc datatype_size.c -o datatype_size

(ashish@kali)-[~/38batch]
$ cat datatype_size.c
# include <stdio.h>

int main() {
    printf("Size of char      : %zu bytes\n", sizeof(char));
    printf("Size of int       : %zu bytes\n", sizeof(int));
    printf("Size of float      : %zu bytes\n", sizeof(float));
    printf("Size of double     : %zu bytes\n", sizeof(double));
    printf("Size of long       : %zu bytes\n", sizeof(long));
    printf("Size of unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("Size of long long  : %zu bytes\n", sizeof(long long));

    return 0;
}

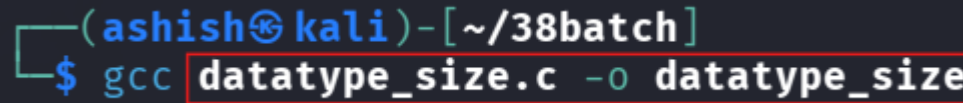
(ashish@kali)-[~/38batch]
$ ./datatype_size
Size of char      : 1 bytes
Size of int       : 4 bytes
Size of float     : 4 bytes
Size of double    : 8 bytes
Size of long      : 8 bytes
Size of unsigned int: 4 bytes
Size of long long : 8 bytes
```

Fig20: Command that is used to allocate the size of different datatypes.

B. Terminal Execution & Verification:

➤ **GCC command to create object:**

gcc datatype_size.c -o datatype_size



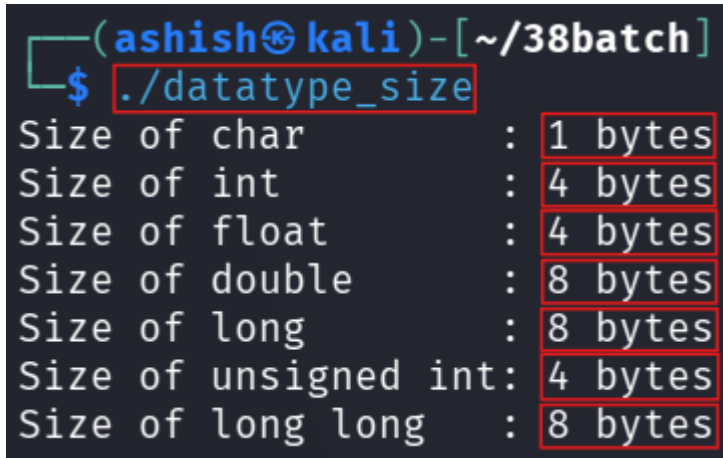
```
(ashish@kali)-[~/38batch]
$ gcc datatype_size.c -o datatype_size
```

Fig21: Command that is used to create objects in C program using **gcc datatype_size.c** file.

C. Check the size of datatype in C:

➤ Gcc command to check size of datatype:

`./gcc observing_virtual_addresses.c -o observing_virtual_addresses`



```
(ashish@kali)-[~/38batch]
$ ./datatype_size
Size of char      : 1 bytes
Size of int       : 4 bytes
Size of float     : 4 bytes
Size of double    : 8 bytes
Size of long      : 8 bytes
Size of unsigned int: 4 bytes
Size of long long : 8 bytes
```

Fig22: Command that is used to allocate the size of different datatypes.

Summary:

Lab 1 will employ `sizeof()` to display the size of `char`, `int`, `float`, `double`, `long`, `unsigned int`, and `long` variables, which will be compared against Table I.

Use the gcc compiler `datatype_size.c -o datatype_size` and execute `datatype_size./`. Note that `sizeof(long)` yields different values (4 and 8 bytes), depending on whether it is a 32-bit or a 64-bit machine.

3. Compare output with Table I.

<u>Data Type size</u>	
<u>Data Type</u>	<u>Size (Bytes)</u>
char	1 byte
int	4 bytes
float	4 bytes
double	8 bytes
long	8 bytes
unsigned int	4 bytes
long long	8 bytes

Fig23: Table which shows different sizes of datatypes.

Summary:

The sizes in the table agree with the 64-bit entries in Table I: char = 1, int = 4, float = 4, double = 8, long = 8, long long = 8, and unsigned int = 4. The system is indeed running LP64, which is common for 64-bit Linux.

Sizeof(long) is different since the C specification leaves it undefined; it depends on the system data model. On 32-bit systems, ILP32 is used, where int, long, and pointers are 4-byte quantities, whereas on 64-bit Linux, LP64 is employed, where long and pointers are 8-byte quantities.

B. Lab 2: Memory Segment Address Mapping

Objective: Observe where different variables live in memory and validate OS memory layout.

Tasks:

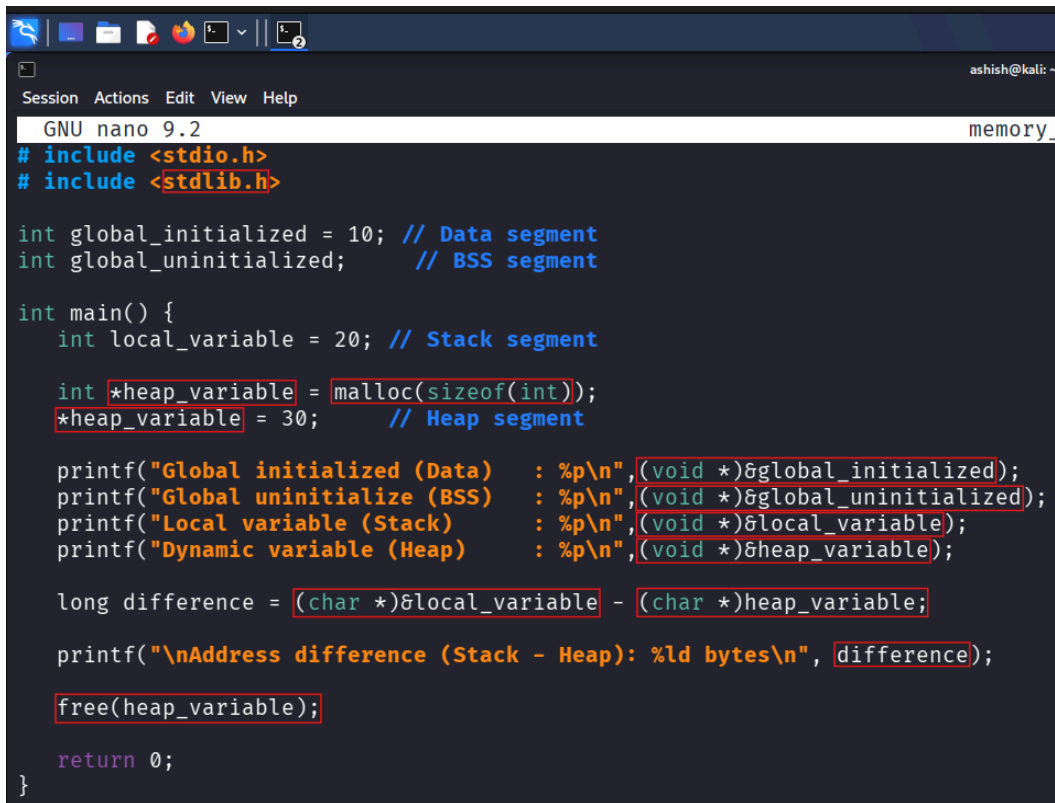
1. Write a program that declares variables in each segment:
 - Global initialized variable (Data segment)
 - Global uninitialized variable (BSS segment)
 - Local variable (Stack segment)
 - Dynamically allocated variable (Heap segment)
2. Print the address of each variable using %p
3. Identify which segment has the highest addresses
4. Identify which segment has the lowest addresses
5. Calculate the address difference between stack and heap

Deliverables:

- Screenshot of output
- Labels identifying each memory segment
- 2-3 sentences explaining what you observed

A. C Source Code:

C program source code which uses Memory Segment Address Mapping.

A screenshot of a terminal window showing a C program being edited in the nano 9.2 text editor. The program is designed to demonstrate memory segment address mapping. It includes headers for stdio.h and stdlib.h. It declares a global initialized variable (Data segment), a global uninitialized variable (BSS segment), a local variable (Stack segment), and a dynamically allocated variable (Heap segment). The program prints the addresses of these variables and calculates the address difference between the stack and heap. The code is as follows:

```
GNU nano 9.2 memory_
# include <stdio.h>
# include <stdlib.h>

int global_initialized = 10; // Data segment
int global_uninitialized;    // BSS segment

int main() {
    int local_variable = 20; // Stack segment

    int *heap_variable = malloc(sizeof(int));
    *heap_variable = 30;     // Heap segment

    printf("Global initialized (Data) : %p\n", (void *)&global_initialized);
    printf("Global uninitialized (BSS) : %p\n", (void *)&global_uninitialized);
    printf("Local variable (Stack) : %p\n", (void *)&local_variable);
    printf("Dynamic variable (Heap) : %p\n", (void *)&heap_variable);

    long difference = (char *)&local_variable - (char *)heap_variable;

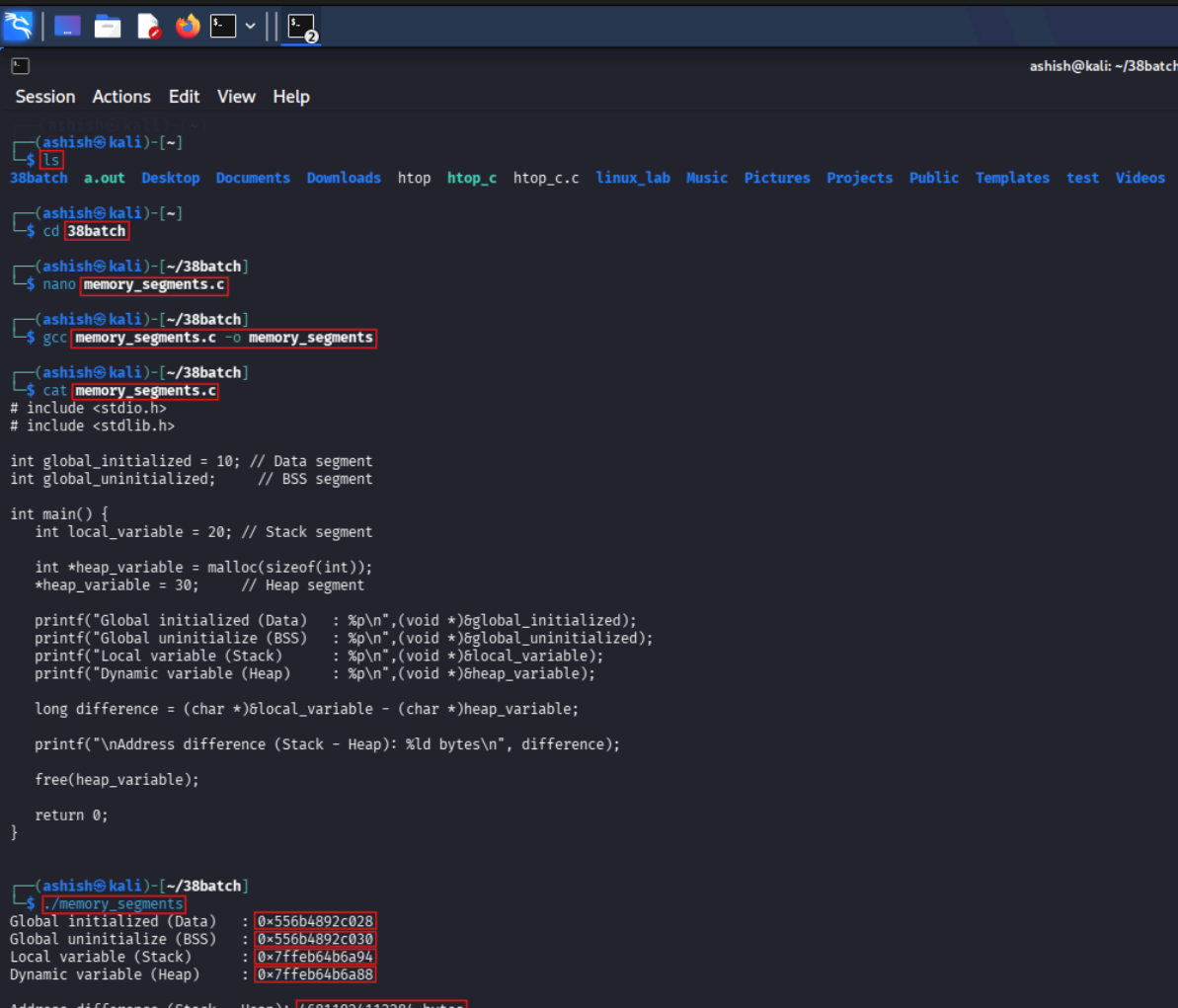
    printf("\nAddress difference (Stack - Heap): %ld bytes\n", difference);

    free(heap_variable);

    return 0;
}
```

Fig24: C source code which uses Memory Segment Address Mapping.

4. File Operation: This command is used to check memory_segments in C programs.



```
ashish@kali: ~/38batch
Session Actions Edit View Help

(ashish@kali)-[~]
$ ls
38batch a.out Desktop Documents Downloads htop htop_c htop_c.c linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano memory_segments.c

(ashish@kali)-[~/38batch]
$ gcc memory_segments.c -o memory_segments

(ashish@kali)-[~/38batch]
$ cat memory_segments.c
# include <stdio.h>
# include <stdlib.h>

int global_initialized = 10; // Data segment
int global_uninitialized;    // BSS segment

int main() {
    int local_variable = 20; // Stack segment

    int *heap_variable = malloc(sizeof(int));
    *heap_variable = 30;     // Heap segment

    printf("Global initialized (Data)   : %p\n", (void *) &global_initialized);
    printf("Global uninitialized (BSS)  : %p\n", (void *) &global_uninitialized);
    printf("Local variable (Stack)       : %p\n", (void *) &local_variable);
    printf("Dynamic variable (Heap)      : %p\n", (void *) &heap_variable);

    long difference = (char *) &local_variable - (char *) &heap_variable;

    printf("\nAddress difference (Stack - Heap): %ld bytes\n", difference);

    free(heap_variable);

    return 0;
}

(ashish@kali)-[~/38batch]
$ ./memory_segments
Global initialized (Data)   : 0x556b4892c028
Global uninitialized (BSS)  : 0x556b4892c030
Local variable (Stack)     : 0x7ffeb64b6a94
Dynamic variable (Heap)    : 0x7ffeb64b6a88

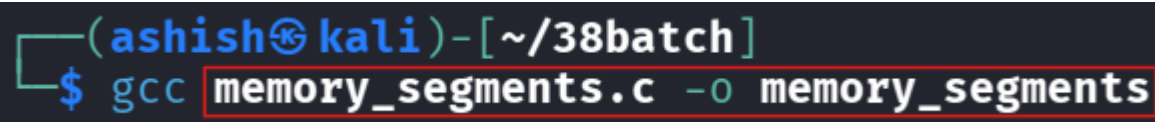
Address difference (Stack - Heap): 46811924113284 bytes
```

Fig25: Command that is used to show output of memory_segment of different data types in C.

B. Terminal Execution & Verification:

➤ GCC command to create object:

gcc memory_segments.c -o memory_segments



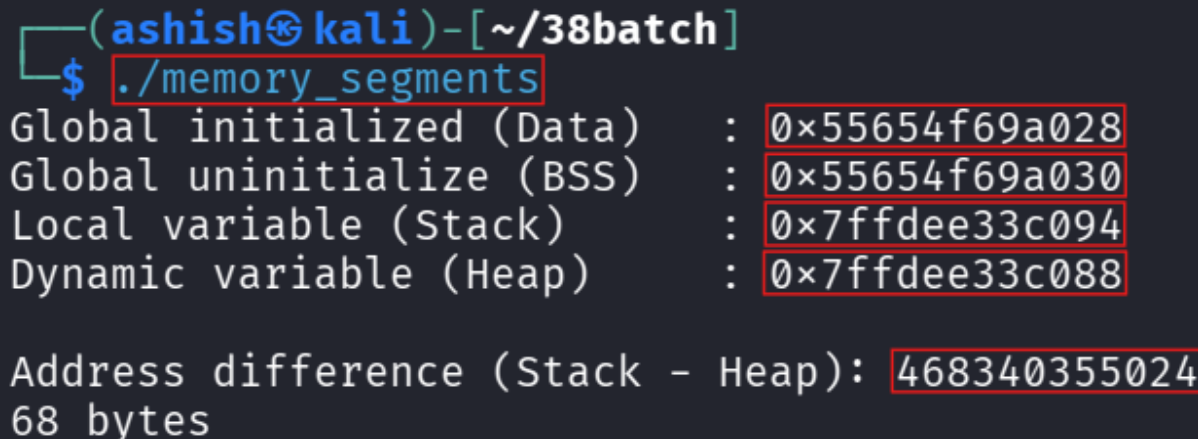
```
(ashish@kali)-[~/38batch]
$ gcc memory_segments.c -o memory_segments
```

Fig26: gcc Command that is used to create objects on a C program using gcc memory_segments.c -o memory_segments.

C. Check the address of datatype in C:

➤ Gcc command to check size of datatype:

`./gcc memory_segments`

A terminal window with a dark background. The prompt is '(ashish@kali)-[~/38batch]'. The command './memory_segments' is entered and executed. The output shows four memory segments with their addresses: 'Global initialized (Data)' at '0x55654f69a028', 'Global uninitialized (BSS)' at '0x55654f69a030', 'Local variable (Stack)' at '0x7ffdee33c094', and 'Dynamic variable (Heap)' at '0x7ffdee33c088'. Below this, it shows the 'Address difference (Stack - Heap): 468340355024' and '68 bytes'. All the hexadecimal addresses and the difference value are highlighted with red boxes.

```
(ashish@kali)-[~/38batch]
$ ./memory_segments
Global initialized (Data)      : 0x55654f69a028
Global uninitialized (BSS)    : 0x55654f69a030
Local variable (Stack)        : 0x7ffdee33c094
Dynamic variable (Heap)       : 0x7ffdee33c088

Address difference (Stack - Heap): 468340355024
68 bytes
```

Fig27: Command that is used to show memory address of different datatypes in C program.

Summary:

Lab 2 looks into memory structure on an OS by declaring variables within each of the Data, BSS, Stack, and Heap segments and displaying their address using %p format. You need to determine the lowest and highest segment and also determine the address gap between Stack and Heap.

To compile, use `gcc memory_segments.c -o memory_segments` and execute the program with `./memory_segments` command (NOT `./gcc memory_segments`). The result demonstrates that Stack occupies the highest address space, Heap the middle one, and Data/BSS occupy the lowest one.

VI. SUMMARY AND KEY TAKEAWAYS:

- I. Data types are contracts that tell the OS how much memory to allocate and how to interpret it.***
- II. Memory is segmented stack for locals, heap for dynamic, data/BSS for globals, text for code. (we will further explore the text section in more details if necessary)***
- III. The OS manages it all by tracking, allocating, protecting, and virtualizing memory.***
- IV. Stack is automatic; heap is manual choice based on lifetime and size needs.***
- V. Virtual memory provides isolation; each process has its own address space.***
- VI. Pointers bridge stack and heap the pointer variable lives on stack, but points to heap memory.***

- 1. Data types are contracts that tell the OS how much memory to allocate and how to interpret it.***

Data types form the contract for the number of bytes that need to be allocated, the way to interpret these bytes (integers, floats, or characters), and the operations that can be performed on these bytes.

The OS needs this contract to allocate memory properly, as validated in labs.

- 2. Memory is segmented stack for locals, heap for dynamic, data/BSS for globals, text for code. (we will further explore the text section in more details if necessary)***

The sections are Stack (local variables, functions), Heap (dynamic memory allocations), Data (initialization of global variables), BSS (uninitialized global variables), and Text (readable-executable codes).

It is not necessary to delve into more details of the text section such as literal strings, function pointers, ASLR, etc.

- 3. The OS manages it all by tracking, allocating, protecting, and virtualizing memory.***

The memory usage is monitored by the OS using page tables and information associated with frames, allocating physical frames on the demand and also protecting with the help of permissions bits.

It also uses virtual memory with the use of MMU to map virtual addresses to physical ones.

4. Stack is automatic; heap is manual choice based on lifetime and size needs.

Stack allocation is automated and done by the compiler, whereas heap allocation is manual, which is decided by the programmer based on the lifetime of the memory (dynamic).

5. Virtual memory provides isolation; each process has its own address space.

Each individual process has its own virtual address space allocated to the physical memory, hence avoiding any conflict.

6. Pointers bridge stack and heap the pointer variable lives on stack, but points to heap memory.

The local pointer variable occupies stack space, but its value is a location in the heap. This is because there is an interconnection between the two segments, making it possible for stack space to access and manage heap space manually.

Knowledge Check Questions:

- I. What is a data type in the context of programming and operating systems?**
- II. Name the five main memory segments in a process memory layout.**
- III. What is the difference between the Data segment and the BSS segment?**
- IV. Why does the OS place the stack at high virtual addresses and the text segment at low addresses?**
- V. What happens to stack memory when a function returns? Who reclaims it?**
- VI. How does the OS know which memory belongs to which process?**

1. What is a data type in the context of programming and operating systems?

A data type is an agreement made between the programmer, the compiler, and the operating system concerning:

- The number of bytes of memory allocated for that data type.
- Interpretation of these bytes (integer, character, etc.).
- Valid operations for that data.

Within programming, it informs the compiler about the handling of the variable. Within the operating system, it provides the amount of memory to be allocated, its representation, and access/privacy.

2. Name the five main memory segments in a process memory layout.

The five major memory segments for a process memory layout include:

- Text (code)
- Data (globals/statics initialized)
- BSS (globals/statics uninitialized)
- Heap (dynamically allocated memory)
- Stack (function call and local variables)

3. What is the difference between the Data segment and the BSS segment?

The data segment holds the initialized global and static variables (e.g. `int x = 10;`), and their initialization values are stored in the executable.

BSS segment contains the uninitialized global and static variables (e.g. `int y;`), which are initialized by OS to zeroes during program initialization. The BSS segment does not hold any values but holds only the size of the segment in the executable file.

4. Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

The stack expands downwards from higher addresses whereas the heap expands upwards from lower addresses this increases sharing of available memory space.

The code segment is placed in lower addresses since it is constant and read only.

5. What happens to stack memory when a function returns? Who reclaims it?

The memory used by the function for local variables gets automatically released after the function executes. This is achieved by simply moving the pointer back to where it was before the function call.

This is done at the program level (not at the operating system level). The data is not wiped; it remains there until some other process uses that space.

6. How does the OS know which memory belongs to which process?

A page table is provided by the OS for each process, where the virtual addresses of the process are mapped to physical memory frames.

During execution, the process's page table is used by the CPU to translate addresses, such that the process sees only its own memory. The OS maintains a record of which frames belong to which process.

VII. CONCLUSION:

The significance of this assignment was that it helped understand how the different data types in C determine memory allocation and differ depending on the architecture, as confirmed by our labs in LP64 architecture and standard memory structure (stack at the top, heap in the middle, and Data/BSS at the bottom). Another important point learned is how processes can be created, executed, coordinated, and destroyed using the aforementioned system calls.